

南開大學
Nankai University

计算机学院

自然语言处理

作业一：文本分类实验报告

姓名：罗雅淇

学号：2313483

2026 年 10 月 12 日

目 录

1 实验概述 **2**

2 数据集与实验设置 **2**
2.1 数据集 2
2.2 数据划分 3
2.3 评价指标 3

3 Task 1：Bag of Words **3**
3.1 特征构造 3
3.2 核心代码 4
3.3 实验结果 5
3.4 结果分析 5

4 Task 2：Word Embedding **6**
4.1 文本的向量表示 6
4.2 三组词向量的来源 6
4.3 核心代码 6
4.4 实验结果 7
4.5 结果分析 8

5 Task 3：Fine-tune BERT **9**
5.1 微调设置 9
5.2 核心代码 9
5.3 实验结果 10
5.4 结果分析 11
5.5 序列长度限制的量化 11

6 综合比较 **12**
6.1 总体结果 12
6.2 表示方式与任务特性的关系 12
6.3 文本长度的影响 12
6.4 评价指标的选择 13

7 实验环境与运行说明 **13**

8 总结 **13**

1 实验概述

本次实验在 NYT 新闻数据集上完成 business、politics、sports 三类文本分类，按作业要求实现并比较三种文本表示方法。Task 1 使用词袋模型，分别考察二值取值与词频取值两种方案；Task 2 使用 100 维词向量，考察预训练 GloVe 以及在 AG News 和 NYT 两份语料上自行训练的 Word2Vec；Task 3 对 BERT 进行微调。前两个 Task 的分类器统一为 Logistic Regression，使比较集中在文本表示环节。

为使六组结果具备可比性，实验中统一了三项设置：数据划分只生成一次并固定复用，随机种子统一取 42，分词统一使用 `nltk.word_tokenize` 并缓存结果。评价指标为 Accuracy 与 Macro-F1，全部在 NYT 测试集上进行测试。

作业仓库链接：https://github.com/lyaqiqi/NKU_NLP_Assigment/tree/main/HW1

2 数据集与实验设置

2.1 数据集

实验使用 NYT 与 AG News 两份英文新闻语料。NYT 提供文本与类别标签，是分类任务的数据来源；AG News 仅提供文本，在本次作业中只用于训练 Word2Vec，不参与监督训练。

NYT 共 11519 条样本，类别分布与篇幅统计见表 2.1。

类别	样本量	占比	平均词数
business	1429	12.4%	649
politics	1451	12.6%	723
sports	8639	75.0%	622
合计	11519	100%	730

表 2.1: NYT 全量数据类别分布

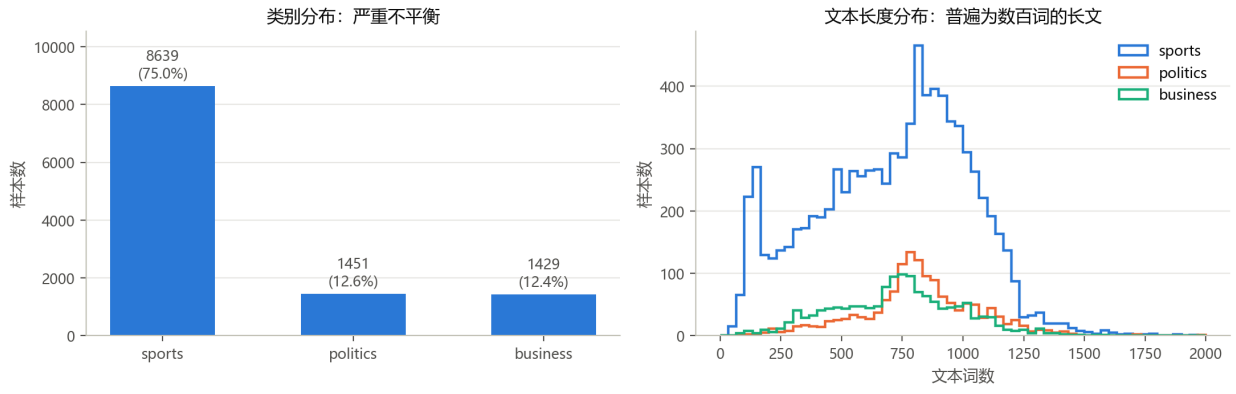


图 2.1: NYT 数据集的类别分布与文本长度分布

该数据集有两项特征对后续实验影响较大。其一是类别分布高度不均衡，sports 占全体的 75%，另外两类合计仅占四分之一，如果全都预测为 sports，Accuracy 可达 0.7500，而 business 与 politics 的 F1 均为 0，Macro-F1 仅为 0.2857，因此评价以 Macro-F1 为主。其二是文本篇幅较长，分词后平均 730 词，中位数 768 词，最长样本接近 6000 词，Task3 选择序列阶段会受到这个特征的影响。

AG News 共 90000 条，篇幅远短于 NYT，平均每条 45 词，全部语料合计 405 万 token。

2.2 数据划分

按作业规定的 8 : 1 : 1 比例划分训练集、验证集与测试集，随机种子取 42。由于类别比例悬殊，划分时采用分层抽样，使三个子集内部的类别构成与全量数据一致，避免少数类在验证集或测试集上数量过少而导致指标波动。

子集	样本量	business	politics	sports
train	9215	1143	1161	6911
val	1152	143	145	864
test	1152	143	145	864

表 2.2: NYT 数据划分（seed=42）

2.3 评价指标

实验报告 Accuracy 与 Macro-F1 两项指标。Accuracy 为预测正确的样本占全部样本的比例：

$$\text{Accuracy} = \frac{\text{预测正确的样本数}}{\text{总样本数}}. \tag{1}$$

Macro-F1 对每个类别单独计算 F1 后取算术平均。对类别 c，设 TP 为正确识别为该类的数量，FP 为其他类别被误判为该类的数量，FN 为该类被误判为其他类别的数量，则

$$\text{Precision}_c = \frac{TP}{TP + FP}, \qquad \text{Recall}_c = \frac{TP}{TP + FN}, \tag{2}$$

$$\text{F1}_c = \frac{2 \cdot \text{Precision}_c \cdot \text{Recall}_c}{\text{Precision}_c + \text{Recall}_c}, \qquad \text{Macro-F1} = \frac{1}{C} \sum_{c=1}^{C} \text{F1}_c. \tag{3}$$

两项指标的差别在于类别权重。Accuracy 以样本为单位统计，样本量大的类别影响更大；Macro-F1 在最后一步取平均时每个类别各占 $1/C$，与样本量无关。

3 Task 1：Bag of Words

3.1 特征构造

词袋模型将 text 表示为词表上的取值向量，不保留词序。实现时先在训练集上统计全部出现过的词，构成词表 $V = \{w_1, \ldots, w_{|V|}\}$，本实验得到 $|V| = 127606$。此后每篇文本均表示为 $|V|$ 维向量 $\mathbf{x} = (x_1, \ldots, x_{|V|})$，其中第 i 维固定对应词 w_i。

两种方案的区别仅在于 x_i 的定义。Binary Bag of Words 只记录词是否出现：

$$x_i = \begin{cases} 1, & w_i \in d, \\ 0, & w_i \notin d, \end{cases} \tag{4}$$

Word Frequency 则记录词在文本中的出现次数：

$$x_i = c_i, \tag{5}$$

其中 d 表示当前文本，c_i 为 w_i 在 d 中出现的次数。

词表仅在训练集上构建。验证集与测试集按同一词表向量化，未登录词直接忽略。若使用全量数据构建词表，测试集特有的词汇也会占据维度，相当于在特征构造阶段引入了测试集信息，评测结果会偏乐观。

为使两种方案的比较不受实现差异干扰，实验中先计算词频矩阵，再在同一矩阵上二值化得到 Binary 版本。这样两者不仅共用词表，列的排列顺序也完全一致，差异被严格限制在取值规则上。

表 3.1 给出两种特征的统计量。词频特征的取值跨度达到 284，单篇文本各维之和的均值为 Binary 的两倍以上，这一量纲差异在第 3.4 节的分析中起到关键作用。

统计量	Binary BoW	Word Frequency
取值范围	$[0, 1]$	$[0, 284]$
单篇文本各维之和的均值	323.4	728.9
训练矩阵稀疏度	0.2535%	
词表规模	127606	

表 3.1: 两种词袋特征的统计量

分类器使用 Logistic Regression，正则强度 C 在 $\{0.01, 0.1, 1, 10\}$ 范围内按验证集 Macro-F1 选取，两种特征各自独立搜索，最终 Binary 取 $C = 1$，词频取 $C = 0.1$。

3.2 核心代码

```
# 文本已用 nltk.word_tokenize 预分词，analyzer 直接返回 token 列表
vectorizer = CountVectorizer(analyzer=lambda toks: toks, min_df=1)

# 词表仅在训练集上构建，另外两个子集沿用同一词表
Xtr_count = vectorizer.fit_transform(train_df["tokens"])   # |V| = 127606
Xva_count = vectorizer.transform(val_df["tokens"])
Xte_count = vectorizer.transform(test_df["tokens"])

# 在同一矩阵上二值化，保证两种特征的词表与列序完全一致
to_bin = lambda X: (X > 0).astype(np.float32)
Xtr_bin, Xva_bin, Xte_bin = to_bin(Xtr_count), to_bin(Xva_count), to_bin(Xte_count)

# Binary BoW
clf_bin = LogisticRegression(C=1.0, max_iter=1000, random_state=42)
clf_bin.fit(Xtr_bin, y_train)

# Word Frequency：同一词表，直接使用计数值
clf_cnt = LogisticRegression(C=0.1, max_iter=1000, random_state=42)
clf_cnt.fit(Xtr_count.astype(np.float32), y_train)
```

方法	划分	Accuracy	Macro-F1	迭代次数
多数类基线	test	0.7500	0.2857	—
Binary BoW	train	1.0000	1.0000	
Binary BoW	val	0.9835	0.9597	30
Binary BoW	test	**0.9887**	**0.9728**	
Word Frequency	train	1.0000	1.0000	
Word Frequency	val	0.9826	0.9621	260
Word Frequency	test	0.9861	0.9712	

表 3.2: Task 1 实验结果

3.3 实验结果

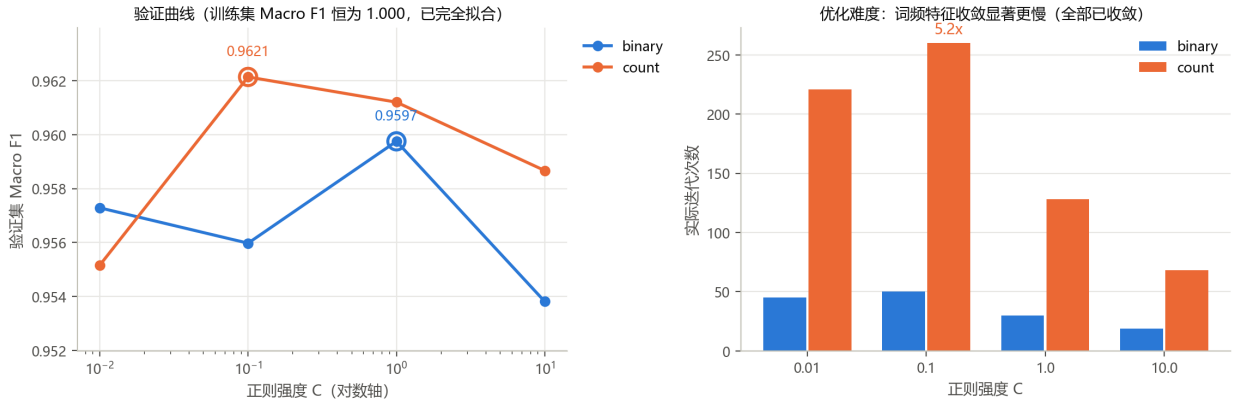


图 3.1: 验证集 Macro-F1 随正则强度的变化，以及两种特征收敛所需的迭代次数

3.4 结果分析

两种特征在训练集上的两项指标都是 1.0000。这一结果在 127606 维的特征空间中属于预期之内，训练样本数远小于特征维数时，线性模型总能找到将各类完全分开的超平面，因此训练集的表现并不能用于区分两种取值方式，需要看验证集和测试集。

测试集上 Binary BoW 的 Accuracy 与 Macro-F1 为 0.9887 和 0.9728，Word Frequency 为 0.9861 和 0.9712；验证集上则是 Word Frequency 的 Macro-F1 更高，0.9621 对 0.9597。两处差距都不超过 0.003，而且方向相反，因此尚不足以判定某一种取值方式更优。

训练开销也有所不同，达到最优结果时，Word Frequency 需要 lbfgs 迭代 260 次，Binary BoW 仅需 30 次，其余三个 C 取值上同样维持三到五倍的关系。词频特征的取值跨越 0 至 284，高频功能词所在维度与主题词所在维度相差一到两个数量级，梯度下降需要更多步数才能收敛；Binary BoW 将所有维度限制在 0 与 1 之间，相当于在构造特征时就完成了尺度归一。在分类性能相近的前提下，这使 Binary BoW 成为更经济的选择。

4 Task 2：Word Embedding

4.1 文本的向量表示

词袋为每个词分配独立维度，维度之间相互正交，无法表达词与词的语义关联。词向量将每个词映射到低维连续空间，使语义或用法相近的词在空间中位置相邻，语义关系因而被编码进坐标本身。

本 Task 统一使用 100 维词向量。文档向量按作业规定的方式计算，即对文本中所有能够查到词向量的单词取算术平均：

$$e(d) = \frac{1}{n} \sum_{i=1}^{n} e(w_i), \tag{6}$$

其中 n 为该文本中有效单词的数量，查不到向量的词不计入平均。无论原文篇幅如何，输出均为固定的 100 维向量，可直接输入 Logistic Regression。

与词袋相比，该表示在两个方面做出取舍。维度由 127606 降至 100，表示更为紧凑，并引入了语义信息；但平均操作同时带来信息损失，一方面词序未被保留，另一方面少数关键词的向量会与文本中其余数百个普通词一同参与平均，其贡献被摊薄。本数据集平均篇幅 730 词，这一摊薄效应较短文本更为明显。

4.2 三组词向量的来源

Pre-trained GloVe. 采用指定的 glove.6B 预训练向量中的 100 维版本。该向量在维基百科与 Gigaword 共 60 亿 token 的语料上训练，收录 40 万词，可直接使用。统计显示其对 NYT 文本的 token 覆盖率为 98.04%。

Word2Vec on AG News. 使用 AG News 的 90000 篇正文训练 Word2Vec，采用 gensim 实现的 CBOW 结构。CBOW 以上下文窗口内的词预测中心词，训练过程仅使用文本，不需要类别标签，因此无标注的 AG News 可以直接作为语料。超参设置为维度 100、窗口 5、最低词频 5、训练 5 轮。过滤后词表为 25389 词，对 NYT 的 token 覆盖率 94.23%，是三组中最低的。

Word2Vec on NYT. 将训练语料替换为 NYT 自身，其余超参与上一组完全相同，使两组之间的唯一变量为语料。NYT 共 841 万 token，约为 AG News 的两倍，过滤后词表 35501 词，token 覆盖率 98.13%。该组使用了全部 NYT 文本，其中包含验证集与测试集的正文。Word2Vec 的训练过程不读取类别标签，因此不构成标签泄漏，作业要求中也表述为使用 NYT 文本数据训练。但严格而言测试集文本参与了表示学习，若需完全避免，应将语料限制在训练集范围内。

4.3 核心代码

```
def build_doc_vectors(tokens_list, lookup, dim=100):
    # 对能查到词向量的词取平均，查不到的跳过
    D = np.zeros((len(tokens_list), dim), dtype=np.float32)
    for i, toks in enumerate(tokens_list):
        vecs = [lookup[w] for w in toks if w in lookup]
        if vecs:
            D[i] = np.mean(vecs, axis=0)
    return D

# 读取 GloVe 时只保留 NYT 词表内的词。完整载入 40 万词约需 2.4 GB 内存，
# 而未出现在 NYT 中的词在本实验中不会被查询，筛除不影响结果。
nyt_vocab = set()
```

```
for t in df["tokens"]:
    nyt_vocab.update(t)
glove = {}
with open("glove.6B.100d.txt", encoding="utf-8") as f:
    for line in f:
        sp = line.find(" ")
        if line[:sp] in nyt_vocab:
            glove[line[:sp]] = np.fromstring(line[sp+1:], dtype=np.float32, sep=" ")

# 两组自训模型使用完全相同的超参，仅训练语料不同
W2V_KW = dict(vector_size=100, window=5, min_count=5, sg=0,   # sg=0 为 CBOW
              workers=4, epochs=5, seed=42)
w2v_ag  = Word2Vec(sentences=ag_tokens, **W2V_KW)              # 词表 25389
w2v_nyt = Word2Vec(sentences=df["tokens"].tolist(), **W2V_KW)  # 词表 35501

for lookup, C in [(glove, 100), (w2v_ag.wv, 10), (w2v_nyt.wv, 1)]:
    D = build_doc_vectors(df["tokens"].tolist(), lookup)
    clf = LogisticRegression(C=C, max_iter=3000, random_state=42)
    clf.fit(D[tr_idx], y_train)
```

4.4 实验结果

方法	词表规模	token 覆盖率	Accuracy	Macro-F1
Pre-trained GloVe	66439	98.04%	**0.9852**	**0.9637**
Word2Vec on AG News	25389	94.23%	0.9809	0.9541
Word2Vec on NYT	35501	98.13%	0.9826	0.9556

表 4.1: Task 2 测试集结果

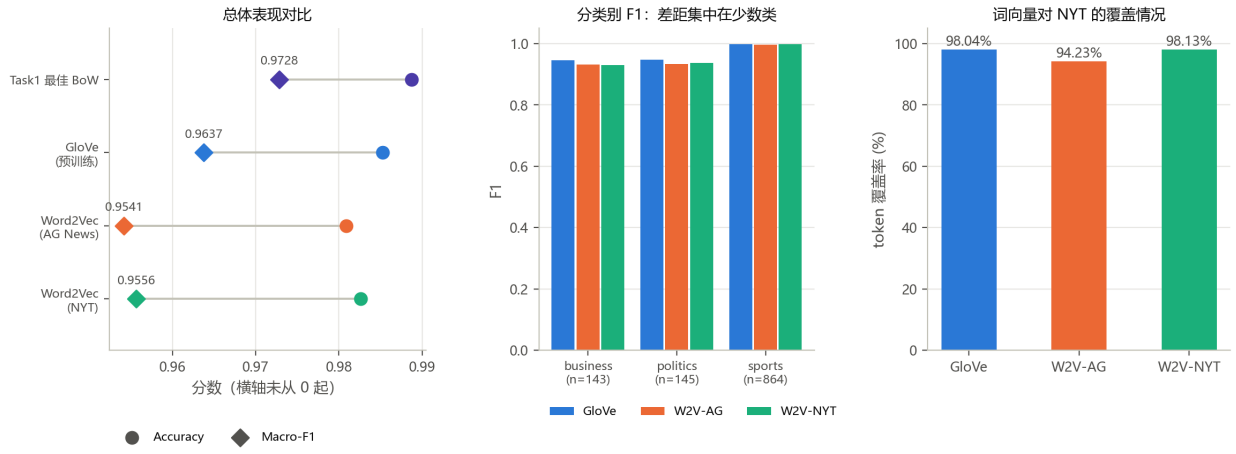


图 4.1: Task 2 三组实验的总体表现、分类别 F1 与词汇覆盖率

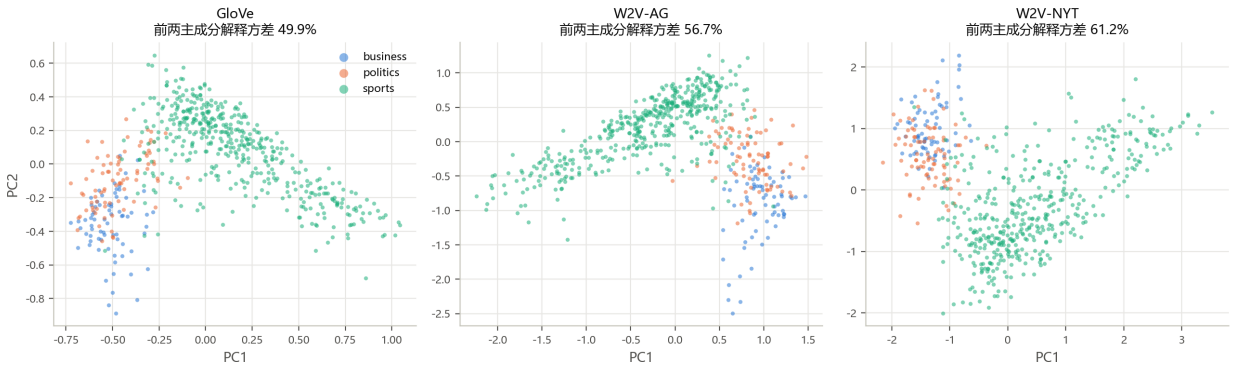


图 4.2: 三种词向量下文本表示的 PCA 投影

4.5 结果分析

三组词向量的 Macro-F1 均在 0.95 以上。其中预训练 GloVe 表现最好，两项指标为 0.9852 与 0.9637；两组自行训练的 Word2Vec 分别为 0.9541 与 0.9556，较 GloVe 低约一个百分点。

GloVe 领先的原因分析如下：首先是语料规模，GloVe 使用 60 亿 token，而自训模型分别仅有 405 万与 841 万，相差二至三个数量级，数百万 token 不足以为词表中的每个词估计出稳定的向量表示。其次是语料规模差异导致所学内容的性质不同，小规模语料更多地记录了该语料内部的搭配习惯，而非词的通用语义。表 4.2 列出同一个词在三套向量中的最近邻，可以直观反映这一区别。

词	词向量	余弦相似度最高的五个词
`bank`	GloVe	banks, banking, credit, investment, financial
	W2V-AG	hispano, santander, banks, kanye, banco
	W2V-NYT	telekom, commission, bloc, i.m.f, bp
`court`	GloVe	judge, appeals, supreme, courts, trial
	W2V-AG	judge, hearing, panel, appeal, jury
	W2V-NYT	courts, ruling, judge, supreme, justices
`football`	GloVe	soccer, basketball, league, rugby, hockey
	W2V-AG	basketball, hockey, coaching, soccer, coaches
	W2V-NYT	basketball, college, hockey, sport, soccer

表 4.2: 同一个词在三套词向量中的最近邻

`bank` 一行的差异最为明显。GloVe 给出的是银行业的通用概念，包括银行、信贷、投资、金融；AG News 训练的向量指向若干具体银行名称；NYT 训练的向量则集中在欧洲债务危机的语境，出现了电信企业、欧盟委员会与国际货币基金组织。这说明在该语料中 bank 常与欧洲央行共同出现，模型学到的是语料内部的共现关系。相比之下 `football` 一行三者差别不大，均为其他球类运动，说明在词汇边界清晰的领域，小规模语料也能得到可用的表示，差距主要出现在一词多义、需要跨语境归纳的情形。

两组自训向量之间，NYT 的 Macro-F1 高出 AG News 0.0015。该差值很小，且实验中观察到 gensim 在多线程训练下结果不完全可复现，即便固定随机种子，两次运行的指标仍有出入。为确定波动范围，实验使用五个不同的随机种子分别重复训练两组模型，AG News 的 Macro-F1 分布于 0.9517 至 0.9590 之间，NYT 分布于 0.9473 至 0.9626 之间，两个区间完全重叠。据此判断，在本实验的超参设置下，更换这两份语料未能带来可区分的性能差异。这一结果可以理解为两份语料的优势相互抵消：AG News 的篇目更多，NYT 的语料总量更大且与分类任务同源。

将三组结果与 Task 1 对照，词向量方法整体低于 Binary BoW，差距介于 0.009 至 0.019 之间。这与” 引入语义信息应当提升效果” 的预期不符。本实验认为主要原因在于平均操作与数据篇幅的交互。文本平均含 730 词，对 730 个向量取平均后，结果趋近于语料的整体均值，不同文本的向量彼此靠拢，

可区分性下降。词袋虽然不包含语义信息，却将每个词的出现情况完整保留在 127606 个维度上，关键词一旦出现即可直接影响对应维度的取值。对于依靠主题词即可区分的新闻分类任务，后者更为有利。

图 4.2 的 PCA 投影给出了直观的印证：三种表示下 sports 均与其余两类明显分离，而 business 与 politics 的样本点始终重叠在一起，说明三组词向量在这两类之间都未能形成清晰的边界。

5 Task 3：Fine-tune BERT

5.1 微调设置

前两个 Task 将文本转换为固定向量后交由独立的分类器处理，BERT 则把文本直接送入预训练网络，在输出端接分类层，表示与分类在同一次反向传播中共同更新。

这带来两点区别。一是词的表示不再固定，GloVe 中同一个词始终对应同一向量，而 BERT 的输出取决于上下文；位置信息经由 position embedding 进入模型，Self-Attention 进一步建模词之间的依赖关系，这些都是前两种表示无法提供的。二是训练方式，微调会调整 BERT 全部 1.1 亿参数，而不仅是末端分类层，预训练阶段获得的语言知识得以随任务目标继续调整。

按作业要求，模型选用 google-bert/bert-base-uncased，序列长度设为 `max_length=64`，训练 3 轮。作业未规定的超参按微调的常规做法设置，列于表 5.1。

超参	取值
优化器	AdamW
学习率	2×10^{-5}
batch size	32
学习率调度	前 10% 步数线性预热，之后线性衰减
weight decay	0.01，bias 与 LayerNorm 不施加
梯度裁剪阈值	1.0

表 5.1: BERT 微调的超参设置

5.2 核心代码

```
tokenizer = AutoTokenizer.from_pretrained("google-bert/bert-base-uncased")
enc = tokenizer(texts, max_length=64, truncation=True,
                padding="max_length", return_tensors="pt")

model = AutoModelForSequenceClassification.from_pretrained(
    "google-bert/bert-base-uncased", num_labels=3).to(DEVICE)

# 微调惯例：bias 与 LayerNorm 两类参数不施加权重衰减
no_decay = ["bias", "LayerNorm.weight"]
grouped = [
    {"params": [p for n, p in model.named_parameters()
                if not any(nd in n for nd in no_decay)], "weight_decay": 0.01},
    {"params": [p for n, p in model.named_parameters()
                if any(nd in n for nd in no_decay)], "weight_decay": 0.0},
]
optimizer = torch.optim.AdamW(grouped, lr=2e-5)
scheduler = get_linear_schedule_with_warmup(
    optimizer, int(0.1 * total_steps), total_steps)

```

```
for ep in range(3):
    model.train()
    for b in train_loader:
        loss = model(**b).loss
        loss.backward()
        torch.nn.utils.clip_grad_norm_(model.parameters(), 1.0)
        optimizer.step(); scheduler.step(); optimizer.zero_grad()
```

5.3 实验结果

epoch	train loss	val loss	val Accuracy	val Macro-F1
1	0.2752	0.1140	0.9644	0.9194
2	0.0507	0.0871	0.9748	0.9444
3	0.0215	0.0963	0.9766	0.9492

表 5.2: BERT 微调的训练过程

类别	Precision	Recall	F1
business	0.9324	0.9650	0.9485
politics	0.9643	0.9310	0.9474
sports	0.9954	0.9954	0.9954
测试集 Accuracy			**0.9835**
测试集 Macro-F1			**0.9637**

表 5.3: BERT 测试集结果

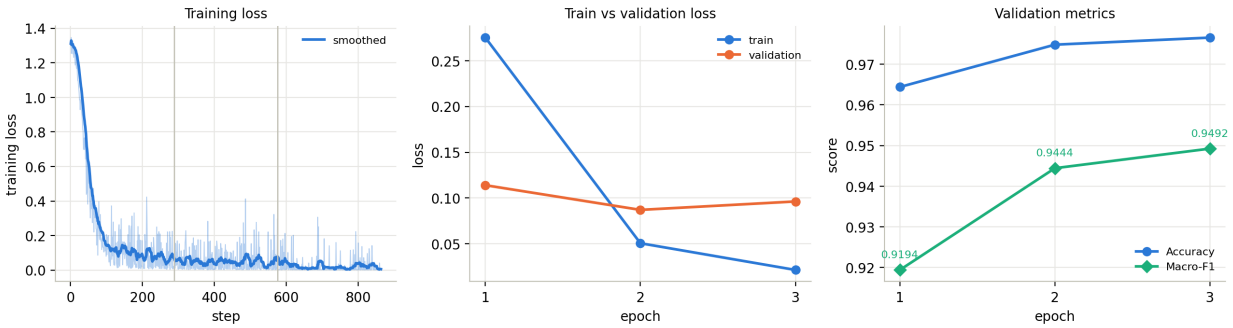


图 5.1: BERT 微调的训练曲线

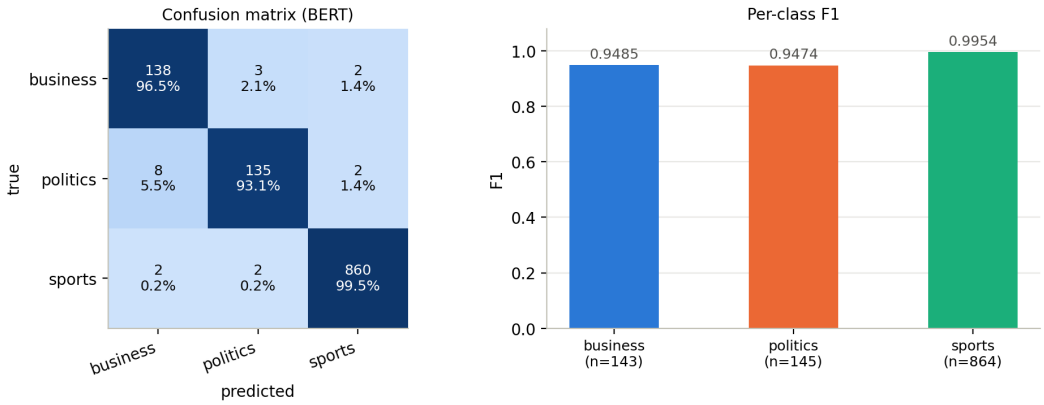


图 5.2: BERT 的混淆矩阵与分类别 F1

5.4 结果分析

从训练过程看，loss 在前 100 步内由 1.3 降至 0.1 以下，收敛速度明显快于从零训练的模型，体现了预训练参数的作用。第 3 轮出现了轻微过拟合的迹象：训练 loss 继续由 0.0507 降至 0.0215，验证 loss 则由 0.0871 回升至 0.0963。同一轮中验证集 Macro-F1 仍有小幅提升，说明模型对已正确分类的样本输出了偏高的置信度，导致损失值上升。

测试集上 BERT 的 Accuracy 为 0.9835，Macro-F1 为 0.9637，低于 Binary BoW 的 0.9887 与 0.9728，与 GloVe 的 Macro-F1 持平。

5.5 序列长度限制的量化

`max_length=64` 对本数据集构成了较强的约束，NYT 新闻平均含 630 个单词，BERT 实际读取的内容不足全文的十分之一，而 Task 1 与 Task 2 使用的是完整正文。不过新闻写作普遍采用倒金字塔结构，核心信息集中于导语部分，这在一定程度上减轻了截断带来的损失。

本实验中 `max_length=64`，NYT 新闻平均含 630 个单词，BERT 实际读取的内容不足全文的十分之一，而 Task 1 与 Task 2 使用的是完整正文。因此，我对文本长度作为变量进行探索，将测试集划分为四段分别统计准确率，结果见表 5.4。

文本长度	样本数	Binary BoW	GloVe	BERT
400 词以下	186	1.0000	0.9839	0.9892
400 至 700 词	272	0.9890	0.9890	**0.9963**
700 至 1100 词	583	0.9863	0.9863	0.9828
1100 词以上	111	0.9820	0.9730	0.9459

表 5.4: 按文本长度分组的测试集准确率

结果呈现清晰的单调趋势。在 400 至 700 词区间，BERT 的准确率为 0.9963，高于其余所有方法，此时 64 个 token 在全文中所占比例相对较高；随着篇幅增加，可见比例持续下降，BERT 的准确率随之降低；在 1100 词以上区间降至 0.9459，较 Binary BoW 低 3.6 个百分点，而另外两种方法在该区间的下降幅度小得多。

6 综合比较

6.1 总体结果

任务	方法	Accuracy	Macro-F1
—	多数类基线	0.7500	0.2857
Task 1	Binary BoW + LR	**0.9887**	**0.9728**
Task 1	Word Frequency + LR	0.9861	0.9712
Task 2	GloVe Mean + LR	0.9852	0.9637
Task 2	Word2Vec-AG Mean + LR	0.9809	0.9541
Task 2	Word2Vec-NYT Mean + LR	0.9826	0.9556
Task 3	BERT-base-uncased	0.9835	0.9637

表 6.1: 各方法在 NYT Test Set 上的结果

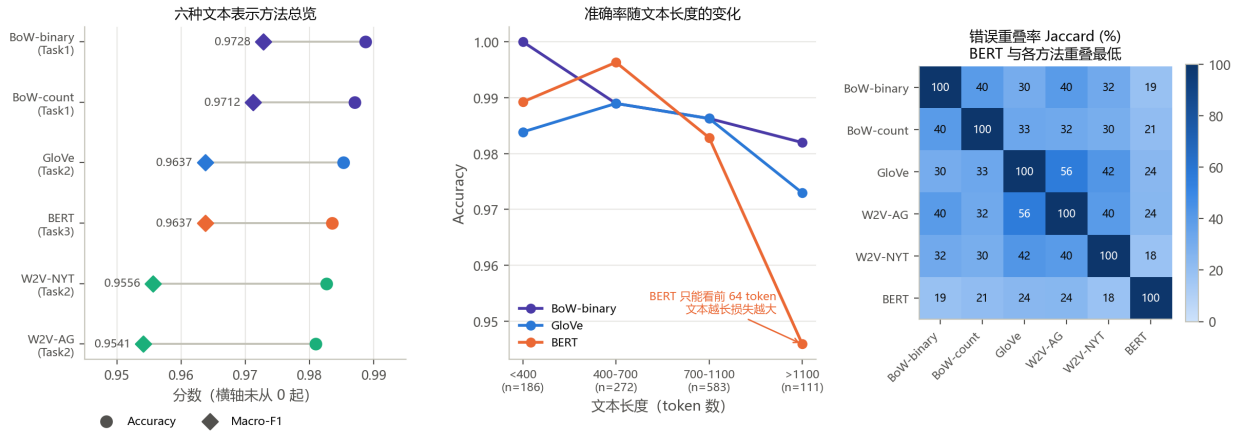


图 6.1: 六种方法的总体对比与准确率随文本长度的变化

六种方法的 Macro-F1 介于 0.9541 至 0.9728 之间，相对多数类基线均有大幅提升。按指标排序，两种词袋方法位居前列，GloVe 与 BERT 次之，两组自训 Word2Vec 排在最后。

6.2 表示方式与任务特性的关系

在六种方法中，维度最高而结构最直接的词袋排在首位。这一名次表明 NYT 三类新闻的区分度主要体现在词汇层面，主题词本身即构成强特征。词袋将每个词的出现情况保留在独立维度上，关键词出现即可直接作用于分类结果；而词向量经过平均后，关键词的贡献被文本中其余词稀释，在本数据集 730 词的平均篇幅下这一损失较为明显。

6.3 文本长度的影响

在 Task 2 中，篇幅决定了平均操作的稀释程度，参与平均的词越多，关键词在文档向量中的权重越低。在 Task 3 中，篇幅决定了 64 个 token 能够覆盖的内容比例，按长度分组的结果显示 BERT 的准确率随篇幅增加而单调下降，在 1100 词以上的区间降幅最为明显。

由此可见，长文本分类中文本表示需要处理的核心问题，是如何在压缩为定长输入的过程中保留判别信息。词袋以高维稀疏的方式完整保留词汇，代价是维度；平均词向量以固定维度换取紧凑性，代

价是信息被稀释；BERT 具备更强的上下文建模能力，但受序列长度限制而无法覆盖全文。

6.4 评价指标的选择

六种方法的 Accuracy 均在 0.98 以上，彼此差距很小，而 Macro-F1 普遍低约 3 个百分点，方法之间的差异也更为明显。拆解到类别可以看到，所有方法在 sports 上的 F1 均超过 0.995，几乎没有区别，全部差距集中于 business 与 politics 两个少数类。若仅报告 Accuracy，这些差异会被占 75% 的多数类掩盖。在类别分布不均衡的数据集上，Macro-F1 能够更准确地反映模型在各类别上的实际表现。

7 实验环境与运行说明

项目	配置
系统	Windows 11
Python	Anaconda 3.12.7
核心库	pandas, scikit-learn, nltk, gensim, transformers, PyTorch
Task 1 与 Task 2	本地 CPU，Intel i5-1240P
Task 3	Google Colab，Tesla T4 GPU
随机种子	42

表 7.1: 主要实验环境

三个 Task 的实现位于仓库的 `HW1/` 目录，均为可独立运行的脚本，按以下顺序执行。

```
python HW1/task1_bow.py          # 词袋模型，约 5 分钟
python HW1/task2_embedding.py    # 词向量，约 4 分钟，复用 Task 1 的分词缓存
python HW1/task3_bert.py         # BERT 微调，GPU 约 12 分钟
```

其中 `common.py` 为三个 Task 共用的模块，负责分词、数据划分与指标计算，各脚本通过它获取划分结果，以保证使用同一批样本。Task 3 在 CPU 上约需 10 小时，实际在 GPU 环境运行；先在本机 CPU 上验证流程，加 `--limit` 参数以小样本运行。

运行 Task 2 前需先下载 GloVe 并解压出 100 维文件：

```
curl -L -o models/glove/glove.6B.zip \
     https://downloads.cs.stanford.edu/nlp/data/glove.6B.zip
cd models/glove && unzip glove.6B.zip glove.6B.100d.txt
```

耗时较长的中间结果，包括分词、网格搜索与 Word2Vec 训练，都会写入 `results/` 目录，再次运行时自动读取，删除对应文件即可强制重算。完整的依赖清单与外部资源的获取方式见 `HW1/README.md`。

8 总结

本次实验在同一份数据与同一组实验设置下完成了三种文本表示方法的实现与比较。词袋模型以词表维度记录词的出现情况或出现次数，词向量将词映射至低维语义空间后取平均得到文本表示，BERT 则通过上下文编码与微调完成端到端分类。

从结果看，Binary BoW 取得最高指标，Accuracy 为 0.9887，Macro-F1 为 0.9728；GloVe 在三组词向量中表现最好；BERT 受 `max_length=64` 的限制未能超过词袋，但在 400 至 700 词的区间内准确率高于其余所有方法。

在本次实验中，我充分意识到文本表示方法的优劣依赖于任务特性，并没有绝对的好坏，本次任务的类别差异主要体现在词汇层面，词袋保留的信息恰好契合这一需求，而语义与上下文信息带来的增益有限，还需承担压缩与截断的代价。